Поиск путей в графе

Плотников Даниил Михайлович

Санкт-Петербургский государственный университет

Оглавление

1. Используемые понятие 4

2. Алгоритм Дейкстры 9

2.1 Постановка задачи 10

2.2 Идея алгоритма 11

2.3 Пример 12

2.5 Ограничения 15

2.6 Реализация 16

2.8 Сложность 18

2.9 Доказательство корректности 19

2.11 Восстановление пути 21

2.13 Практические задачи 23

3. Алгоритм Беллмана-Форда 24

3.1 Постановка задачи 25

3.2 Идея алгоритма 26

3.3 DP постановка 27

3.4 Пример 28

3.5 Оптимизация 29

3.6 Проверка на отрицательные циклы 30

3.7 Сложность 31

3.8 Восстановление пути 32

3.11 Когда использовать 35

3.12 Практические задачи 36

4. Алгоритм Флойда-Уоршелла 37

4.1 Постановка задачи 38

4.2 Идея алгоритма 39

4.3 DP постановка 40

4.4 Переход 41

4.5 Пример 42

4.6 Реализация 43

4.7 Сложность 44

4.8 Отрицательные циклы 45

4.9 Восстановление пути 46

4.12 Практические задачи 49

4.13 Сравнение алгоритмов и их применение 50

Используемые понятие

Базовые определения

Граф — совокупность множеств G(V, E), где

- V (Vertex) — множество вершин
- E (Edge) — множество рёбер

Ориентированный граф — граф, где каждое ребро имеет направление (от u к v). Путь A -> B означает ребро из A в B.

Взвешенный граф — граф, где каждому ребру присвоено числовое значение (вес). Вес ребра может быть отрицательным.

Путь — последовательность вершин $v_0, v_1, ..., v_k$ такая, что для каждого i существует ребро $v_i \to v_{\{i+1\}}$. Обозначается $v_0 \to v_1 \to ... \to v_k$ (например, $A \to B \to C$ означает рёбра $A \to B$ и $B \to C$).

Простой путь — путь без повторяющихся вершин.

Цикл — путь, начинающийся и заканчивающийся в одной вершине. Отрицательный цикл — цикл с суммарным отрицательным весом.

Представление графа

Способ	Описание
Список смежности	Для каждой вершины храним список её соседей. В взвешенном графе — пары (сосед, вес)
Матрица смежности	Матрица $V \times V$, где $A[i][j]$ = вес ребра $i \to j$ (или 1/0 для невзвешенного)

Для алгоритмов на разреженных графах лучше использовать список смежности ($O(V + E)$ памяти). Для плотных — матрицу смежности ($O(V^2)$ памяти).

Обозначения

В алгоритмах поиска кратчайших путей используются:

- s — источник (стартовая вершина)
- dist[v] — текущее известное расстояние от источника до вершины v
- used[v] / processed[v] — посещена ли вершина v
- w(u, v) — вес ребра из u в v
- ∞ — бесконечность (нет пути)
- n — количество вершин

Подробнее о базовых определениях, типах графов и способах их хранения см. в лекции “Теория графов. Введение”.

Алгоритм Дейкстры

Постановка задачи

Задача о кратчайших путях с одним источником (Single-Source Shortest Paths, SSSP):

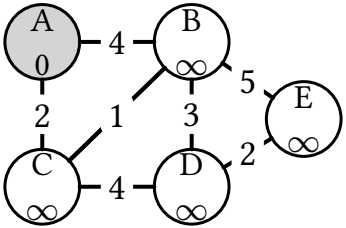
Дан взвешенный ориентированный граф $G(V, E)$ с весами рёбер $w : E \to \mathbb{R}$ и выделенная вершина s. Для каждой вершины $v \in V$ требуется найти длину кратчайшего пути от s к v.

Идея алгоритма

Алгоритм Дейкстры работает по принципу “жадного” выбора: мы поддерживаем множество вершин U, для которых уже вычислены длины кратчайших путей до них из s. На каждой итерации основного цикла выбирается вершина $u \notin U$, которой на текущий момент соответствует минимальная оценка кратчайшего пути. Вершина u добавляется в множество U и производится релаксация всех исходящих из неё рёбер.

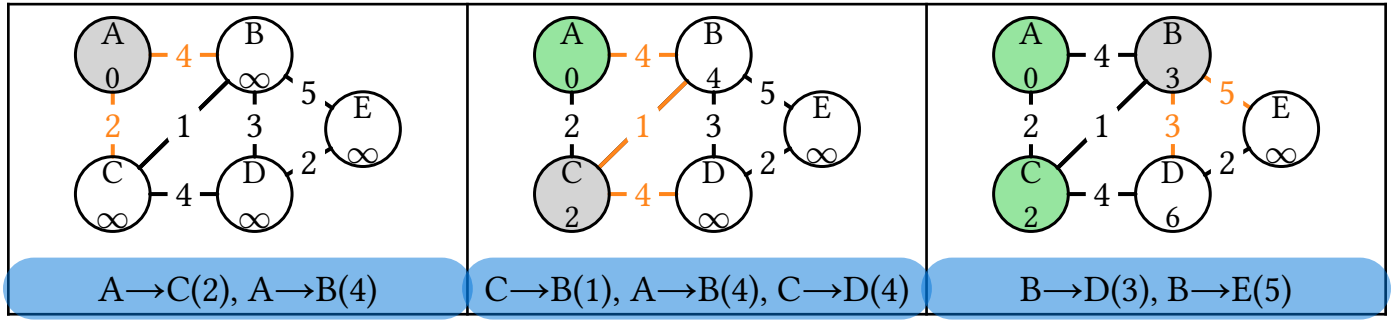
Пример

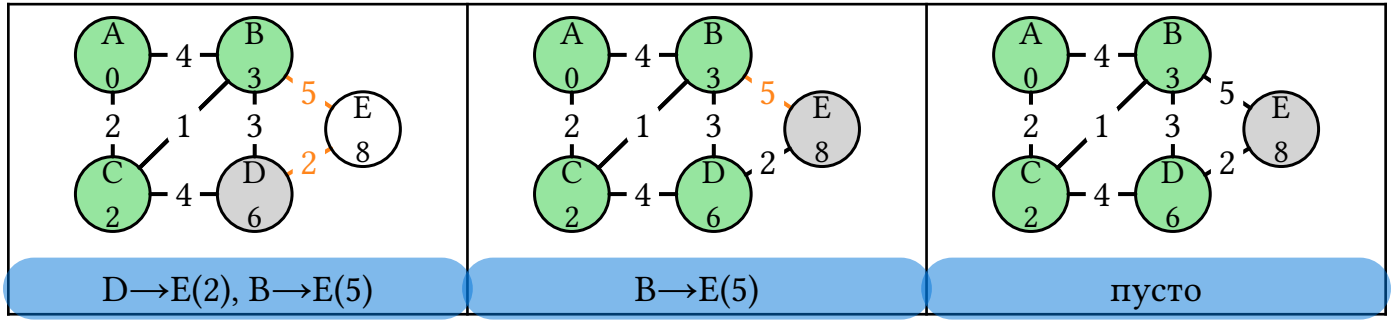
Рассмотрим граф:



Начнём из вершины A (источник).

Пример: пошаговая демонстрация



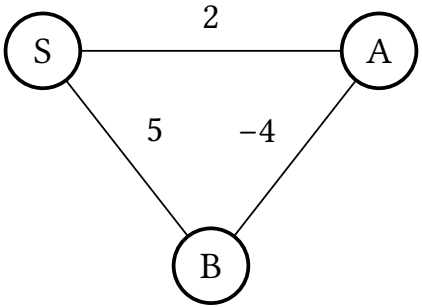


Вершина	A	B	C	D	E
Расстояние от A	0	3	2	6	8

Ограничения

Алгоритм Дейкстры работает только с неотрицательными весами рёбер!

При наличии отрицательных весов алгоритм может дать неверный ответ.



Алгоритм Дейкстры даст $S \to B = 5$, но правильный ответ: $2 + (-4) = -2$

Реализация

```
vector<int> dist(n, INF);
vector<bool> used(n, false);
priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int,
int>>> pq;
```

В коде используется `greater<>` для создания min-heap вместо max-heap (по умолчанию в C++).

Альтернатива `greater<>` — хранить отрицательные значения: `pq.push({-dist[v], v})`.

```
dist[s] = 0;
pq.push({0, s});
while (!pq.empty()) {
    auto [d, v] = pq.top();
    pq.pop();
    if (used[v]) continue;
    used[v] = true;
    for (auto [to, w] : g[v]) {
        if (dist[v] + w < dist[to]) {
            dist[to] = dist[v] + w;
            pq.push({dist[to], to});
        }
    }
}
```

Сложность

- **Временная сложность**: $O((V + E)\log V)$ — каждая вершина извлекается из очереди 1 раз, каждое ребро рассматривается 1 раз
- **Пространственная сложность**: $O(V + E)$ для хранения графа и расстояний

В плотном графе ($E \approx V^2$) сложность можно улучшить до $O(V^2)$ используя обычный массив вместо priority queue.

Доказательство корректности

Теорема. Пусть $w \geq 0$ для всех рёбер. После извлечения вершины v из приоритетной очереди, $d(v) = d^*(s, v)$, где d^* — длина кратчайшего пути.

Доказательство. Индукция по количеству извлечённых вершин.

База. После первой итерации $d(s) = 0 = d^*(s, s)$.

Шаг. Пусть после k итераций для всех извлечённых вершин $d = d^*$. Рассмотрим $(k+1)$-ю вершину v. Все вершины в очереди имеют $d \geq d(v)$.

Возьмём кратчайший путь $P = s \to \ldots \to x \to v$. Вершины на P кроме v уже извлечены. Пусть x — предпоследняя. Тогда x обработана, и алгоритм установил $d(v) \leq d(x) + w = d^*(s, x) + w = d^*(s, v)$.

Так как $d(v)$ минимально в очереди, $d(v) \geq d^*(s, v)$. Следовательно, $d(v) = d^*(s, v)$.

Ключевое свойство: при неотрицательных весах, если мы достаём вершину с минимальным distance из очереди, то никакой другой путь не может дать меньшее расстояние — потому что любой другой путь должен пройти через вершину с расстоянием не меньше текущей.

Восстановление пути

Для восстановления пути храним массив предков `parent`. При релаксации ребра $(v \to u)$, если `dist[v] + w < dist[to]`, устанавливаем `parent[to] = v`. После завершения алгоритма путь от s к t восстанавливается двигаясь от t к s по массиву `parent` и разворачиванием.

```
vector<int> parent(n, -1);
if (dist[v] + w < dist[to]) {
    dist[to] = dist[v] + w;
    parent[to] = v;
    pq.push({dist[to], to});
}
vector<int> get_path(int t) {
    if (dist[t] == INF) return {};
    vector<int> path;
    for (int v = t; v != -1; v = parent[v])
        path.push_back(v);
    reverse(path.begin(), path.end());
    return path;
}
```

Восстановление одного пути требует $O(L)$ времени и $O(V)$ дополнительной памяти.

Практические задачи

- **Network Delay Time**: LeetCode 743
- **Find the City With the Smallest Number of Neighbors**: LeetCode 1334
- **Min Cost to Reach Destination**: LeetCode 1928
- **Кратчайший путь**: acmp.ru 164

Алгоритм Беллмана-Форда

Постановка задачи

Задача о кратчайших путях с одним источником для графов с произвольными весами (в том числе отрицательными):

Дан взвешенный ориентированный граф $G(V, E)$ с весами рёбер $w : E \to \mathbb{Z}$ и выделенная вершина s. Для каждой вершины $v \in V$ требуется найти длину кратчайшего пути от s к v.

В отличие от алгоритма Дейкстры, алгоритм Беллмана-Форда работает с произвольными весами рёбер.

Идея алгоритма

Основная идея — динамическое программирование по количеству рёбер в пути.

Кратчайший путь без циклов (простой путь) содержит не более $|V| - 1$ рёбер.

Если есть отрицательный цикл, достижимый из источника, то кратчайший путь не определён.

DP постановка

Построим динамику:

- Состояние: $\mathrm{dp}_{k(v)}$ — длина кратчайшего пути от s к v, содержащего не более k рёбер
- База DP: $\mathrm{dp}_0(s) = 0$, $\mathrm{dp}_0(v) = +\infty$ для $v \neq s$
- Переход:

Чтобы найти путь к v за $k+1$ ребро, рассмотрим последнее ребро (u, v):

Не использовать ребро (u, v):
$\mathrm{dp}_{k-1}(v)$

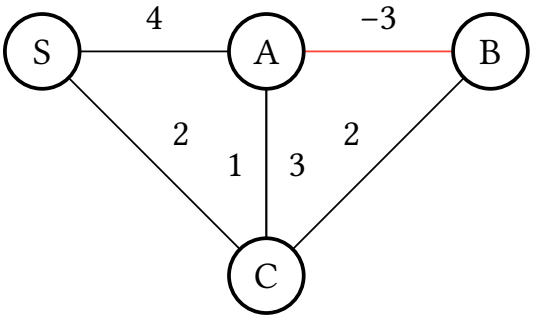
Использовать ребро (u, v):
$\mathrm{dp}_{k-1}(u) + w(u, v)$

Рекуррентное соотношение:

$$\mathrm{dp}_k(v) = \min\Big(\mathrm{dp}_{k-1}(v), \min_{u:\ (\mathrm{u,v})\ \in E} \mathrm{dp}_{k-1}(u) + w(u, v)\Big)$$

Пример

Начнём из вершины S (источник).



k / v	S	A	B	C
0	0	∞	∞	∞
1	0	4	1	2
2	0	3	1	2
3	0	3	1	2

На самом деле, после итерации 2 ничего не меняется, потому что кратчайший путь содержит 2 ребра.

Оптимизация

Заметим, что нам не нужно хранить всю таблицу — достаточно только текущего и предыдущего состояния.

```
vector<long long> dist(n, INF);
dist[s] = 0;
for (int i = 0; i < n - 1; ++i) {
    for (auto [u, v, w] : edges) {
        if (dist[u] != INF && dist[u] + w < dist[v]) {
            dist[v] = dist[u] + w;
        }
    }
}
```

После $|V| - 1$ итерации все кратчайшие пути найдены (если нет отрицательных циклов).

Проверка на отрицательные циклы

После основных итераций делаем ещё одну проверку:

```
for (int i = 0; i < n; ++i) {
    for (auto [u, v, w] : edges) {
        if (dist[u] != INF && dist[u] + w < dist[v]) {
            // Отрицательный цикл обнаружен!
            dist[v] = -INF;
        }
    }
}
```

Если на $(|V|)$-й итерации происходит улучшение — есть отрицательный цикл, достижимый из s.

Сложность

- **Временная сложность**: $O(Vc \cdot E) - |V| - 1$ итерация, на каждой просматриваем все E рёбер
- **Пространственная сложность**: $O(V)$ для хранения расстояний

Восстановление пути

Для восстановления пути храним массив `parent`. Инициализируем `parent[s] = -1`, для остальных вершин `parent[v] = -1`. При релаксации ребра $(u \to v)$, если `dist[u] + w < dist[v]`, обновляем `dist[v]` и записываем `parent[v] = u`. После завершения алгоритма путь от s к вершине t восстанавливается двигаясь от t к s по массиву `parent`.

```
vector<int> parent(n, -1);

for (auto [u, v, w] : edges) {
    if (dist[u] != INF && dist[u] + w < dist[v]) {
        dist[v] = dist[u] + w;
        parent[v] = u;
    }
}
```

```
vector<int> get_path(int t) {
    if (dist[t] == INF) return {};
    vector<int> path;
    for (int v = t; v != -1; v = parent[v])
        path.push_back(v);
    reverse(path.begin(), path.end());
    return path;
}
```

При наличии отрицательного цикла, достижимом из s, некоторые вершины могут получить значение $-\infty$, и путь до них неоднозначен.

Когда использовать

Алгоритм Дейкстры	**Алгоритм Беллмана-Форда**
$O((V+E)\log V)$	$O(Vc \cdot E)$
Только неотрицательные веса	Произвольные веса
Проще, быстрее	Медленнее, но универсальнее
Нет проверки циклов	Может detect negative cycles

Если все веса неотрицательны — используйте Дейкстру. Если есть отрицательные веса — Беллман-Форд.

Практические задачи

- **Network Delay Time**: LeetCode 743
- **Shortest Path in Binary Matrix**: LeetCode 1293
- **Отрицательный цикл**: acmp.ru 175

Алгоритм Флойда-Уоршелла

Постановка задачи

Задача о кратчайших путях для всех пар (All-Pairs Shortest Paths, APSP):

Дан взвешенный ориентированный граф $G(V, E)$ с весами рёбер $w : E \to \mathbb{Z}$. Для каждой пары вершин (u, v) требуется найти длину кратчайшего пути от u к v.

В отличие от алгоритмов Дейкстры и Беллмана-Форда, алгоритм Флойда-Уоршелла находит кратчайшие пути между всеми парами вершин за один запуск.

Идея алгоритма

Основная идея — динамическое программирование по промежуточным вершинам.

DP постановка

Построим динамику:

- Состояние: $\mathrm{dp}_{k(i,j)}$ — длина кратчайшего пути от i к j, где все промежуточные вершины принадлежат множеству $\{1, 2, ..., k\}$
- База DP: $\mathrm{dp}_0(i, j)$ — длина кратчайшего пути без промежуточных вершин (только direct edge или ∞)
- Переход: ?

Переход

Путь от i к j с промежуточными вершинами из множества $\{1..k\}$:

Не использовать вершину k:
$\mathrm{dp}_{k-1}(i, j)$

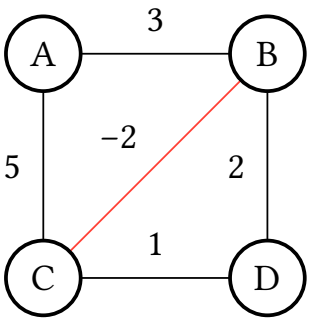
Использовать вершину k:
$\mathrm{dp}_{k-1}(i, k) + \mathrm{dp}_{k-1}(k, j)$

Рекуррентное соотношение:

$$\mathrm{dp}_k(i, j) = \min(\mathrm{dp}_{k-1}(i, j), \mathrm{dp}_{k-1}(i, k) + \mathrm{dp}_{k-1}(k, j))$$

Пример

Рассмотрим граф:



Начальная матрица:

	A	B	C	D
A	0	3	5	∞
B	∞	0	∞	2
C	∞	-2	0	1
D	∞	∞	∞	0

Реализация

```
const long long INF = 1e18;
vector<vector<long long>> dist(n, vector<long long>(n, INF));

for (int i = 0; i < n; ++i) dist[i][i] = 0;
for (auto [u, v, w] : edges) dist[u][v] = w;

for (int k = 0; k < n; ++k)
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            if (dist[i][k] + dist[k][j] < dist[i][j])
                dist[i][j] = dist[i][k] + dist[k][j];
```

Три вложенных цикла — это и есть алгоритм Флойда-Уоршелла. Очень компактный!

Сложность

- **Временная сложность**: $O(V^3)$ — три вложенных цикла по V
- **Пространственная сложность**: $O(V^2)$ для матрицы расстояний

Для плотных графов (много рёбер) — Флойд-Уоршелл. Для разреженных — Дейкстра V раз.

Отрицательные циклы

Алгоритм Флойда-Уоршелла также может обнаруживать отрицательные циклы!

После выполнения, если $\mathrm{dp}_{n(i,i)} < 0$ для какого-то i, то в графе есть отрицательный цикл, достижимый из i.

```
bool hasNegativeCycle = false;
for (int i = 0; i < n; ++i) {
    if (dist[i][i] < 0) {
        hasNegativeCycle = true;
        break;
    }
}
```

Восстановление пути

Для восстановления конкретного пути храним матрицу `next`. Изначально, если есть ребро $i \to j$, полагаем `next[i][j] = j`, иначе `next[i][j] = -1`. При релаксации через k: если `dist[i][k] + dist[k][j] < dist[i][j]`, то обновляем `dist[i][j]` и устанавливаем `next[i][j] = next[i][k]`.

```
vector<vector<int>> next(n, vector<int>(n, -1));
for (int i = 0; i < n; ++i)
    for (int j = 0; j < n; ++j)
        if (dist[i][j] != INF && i != j)
            next[i][j] = j;

for (int k = 0; k < n; ++k)
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            if (dist[i][k] + dist[k][j] < dist[i][j]) {
                dist[i][j] = dist[i][k] + dist[k][j];
                next[i][j] = next[i][k];
            }
```

```
vector<int> get_path(int i, int j) {
    if (dist[i][j] == INF) return {};
    if (next[i][j] == -1) return {};
    vector<int> path;
    int cur = i;
    while (cur != j) {
        path.push_back(cur);
        cur = next[cur][j];
    }
    path.push_back(j);
    return path;
}
```

Матрица `next` добавляет $O(V^2)$ памяти, что совпадает с пространственной сложностью алгоритма.

Практические задачи

- **Shortest Path Distance Between All Pairs**: LeetCode 1334
- **Evaluate Division**: LeetCode 399 (похожая задача)
- **Флойд-Уоршелл**: acmp.ru 175

Сравнение алгоритмов и их применение

Алгоритм	Время	Отриц. веса	Примечание
Дейкстра	$O((V+E)\log V)$	только ≥ 0	быстрый, для разреженных графов
Беллман-Форд	$O(V \cdot E)$	да	универсальный, находит циклы
Флойд-Уоршелл	$O(V^3)$	да	все пары, плотные графы

Выводы:

- Неотрицательные веса $\longrightarrow$ Дейкстра.
- Отрицательные веса $\longrightarrow$ Беллман-Форд.
- Все пары $\longrightarrow$ Флойд-Уоршелл.